

THUẬT TOÁN DIJKSTRA

Tìm đường đi ngắn nhất — Hướng dẫn giải chi tiết từng bước

1. Thuật toán Dijkstra dùng để làm gì?

Thuật toán Dijkstra (đọc là “Đi-cờ-xtra”) tìm **đường đi ngắn nhất** từ một đỉnh xuất phát (gọi là **đỉnh nguồn**) đến tất cả các đỉnh còn lại trong một đồ thị có trọng số. “Trọng số” chính là chi phí / quãng đường / thời gian để đi qua mỗi cạnh.

Một số ứng dụng quen thuộc: tìm đường trên bản đồ (Google Maps), định tuyến gói tin trong mạng máy tính, tìm chuyến bay rẻ nhất, hay tìm số bước ít nhất trong các trò chơi.

⚠ Điều kiện quan trọng

Dijkstra chỉ hoạt động đúng khi **mọi trọng số đều không âm** (lớn hơn hoặc bằng 0). Nếu đồ thị có cạnh âm, ta phải dùng thuật toán khác (ví dụ Bellman-Ford).

2. Bốn khái niệm cần nắm trước

- **$d(v)$ — khoảng cách tạm thời.** Là độ dài đường đi ngắn nhất *tìm được cho tới hiện tại* từ nguồn đến đỉnh v . Ban đầu chưa biết gì nên đặt bằng ∞ (vô cùng), riêng đỉnh nguồn bằng 0.
- **Đỉnh trước (prev).** Ghi nhớ ta đã đi tới v từ đỉnh nào. Nhờ nó, sau khi chạy xong ta truy ngược lại được toàn bộ đường đi, chứ không chỉ biết mỗi độ dài.
- **Đỉnh đã chốt.** Là đỉnh mà khoảng cách $d(v)$ đã **chắc chắn đúng**, không thể ngắn hơn được nữa. Một khi đã chốt thì không bao giờ xét lại.
- **Nới lỏng (relaxation).** Đây là thao tác cốt lõi: đứng tại đỉnh u vừa chốt, ta thử đi tiếp sang đỉnh kề v . Nếu **$d(u) + \text{trọng số cạnh } (u,v) < d(v)$** thì ta đã tìm được đường tốt hơn nên cập nhật lại $d(v)$ và đặt $\text{prev}(v) = u$.

3. Thuật toán chạy như thế nào?

Toàn bộ thuật toán chỉ gồm 4 ý:

- 1 **Khởi tạo:** $d(\text{nguồn}) = 0$, mọi đỉnh khác $= \infty$. Chưa đỉnh nào được chốt.
- 2 **Chọn:** trong các đỉnh *chưa chốt*, lấy đỉnh có d nhỏ nhất rồi chốt nó lại.
- 3 **Nới lỏng:** từ đỉnh vừa chốt, cập nhật khoảng cách cho mọi đỉnh kề của nó.
- 4 **Lặp lại** bước 2 và 3 cho đến khi tất cả các đỉnh đều được chốt.

Mã giả (pseudo-code):

KHỞI TẠO $d(\text{nguồn}) = 0$; $d(\text{các đỉnh khác}) = \infty$

Đưa tất cả các đỉnh vào tập “chưa chốt”

TRONG KHI tập “chưa chốt” vẫn còn đỉnh:

u = đỉnh chưa chốt có $d(u)$ **nhỏ nhất**

chốt u (đánh dấu đã xong)

VỚI MỖI đỉnh v kề với u :

nếu $d(u) + w(u,v) < d(v)$:

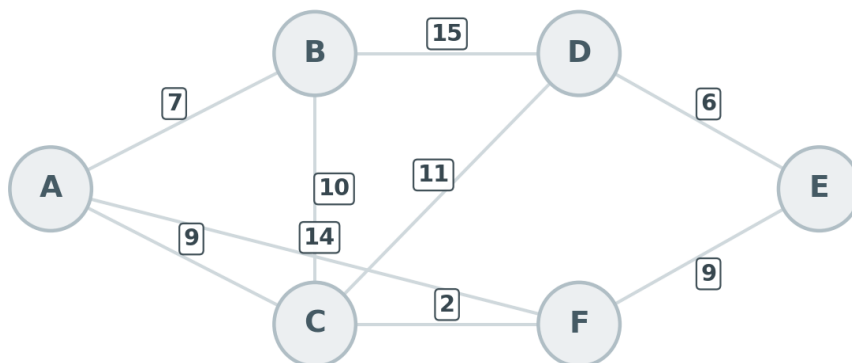
$d(v) = d(u) + w(u,v)$ ← nói lỏng

$\text{prev}(v) = u$

4. Bài toán ví dụ

Cho đồ thị vô hướng dưới đây gồm 6 đỉnh A, B, C, D, E, F. Mỗi cạnh ghi kèm trọng số. **Yêu cầu:** tìm đường đi ngắn nhất từ đỉnh **A** tới mọi đỉnh còn lại.

Đồ thị ban đầu — tìm đường đi ngắn nhất từ đỉnh A



Hình 1. Đồ thị ban đầu của bài toán.

Danh sách các cạnh và trọng số:

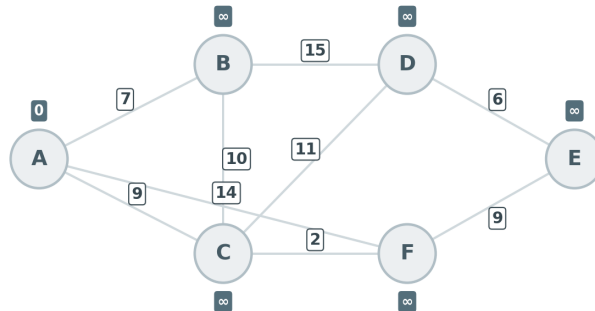
Cạnh	Trọng số	Cạnh	Trọng số	Cạnh	Trọng số
A - B	7	B - D	15	C - F	2
A - C	9	C - D	11	D - E	6
A - F	14	B - C	10	E - F	9

5. Lời giải chi tiết từng bước

Khởi tạo

Đặt $d(A) = 0$ vì đứng tại nguồn thì quãng đường bằng 0. Tất cả các đỉnh còn lại chưa biết nên để bằng ∞ . Chưa có đỉnh nào được chốt.

Khởi tạo: $d(A)=0$, các đỉnh còn lại = ∞

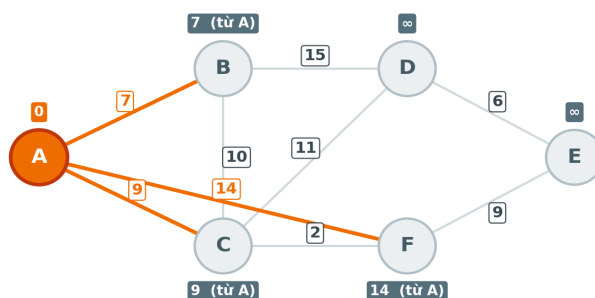


Đỉnh	A	B	C	D	E	F
$d(v)$	0	∞	∞	∞	∞	∞
Đỉnh trước	-	-	-	-	-	-
Trạng thái	chờ	chờ	chờ	chờ	chờ	chờ

Bước 1 — Chốt A

Đỉnh chưa chốt có d nhỏ nhất là **A** ($d = 0$) nên ta chốt A. Từ A có 3 cạnh: A-B (7), A-C (9), A-F (14). Cả ba đỉnh B, C, F đang là ∞ nên đều được cập nhật: $d(B)=7$, $d(C)=9$, $d(F)=14$. Đỉnh trước của cả ba là A.

Bước 1 — Chọn A ($d=0$), nối lỏng các cạnh từ A



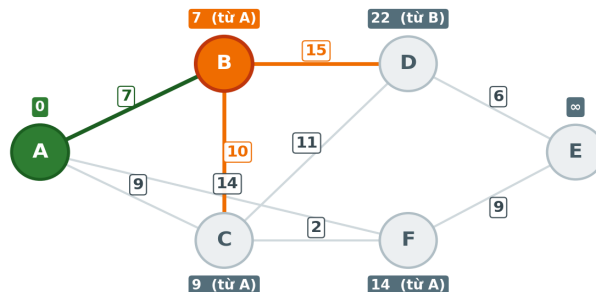
Cập nhật: $d(B)=7$, $d(C)=9$, $d(F)=14$

Đỉnh	A	B	C	D	E	F
$d(v)$	0	7	9	∞	∞	14
Đỉnh trước	-	A	A	-	-	A
Trạng thái	đã chốt ✓	chờ	chờ	chờ	chờ	chờ

Bước 2 — Chốt B

Trong các đỉnh chưa chốt {B, C, D, E, F}, nhỏ nhất là **B** ($d = 7$) → chốt B. Từ B nói lỏng: tới D được $7 + 15 = 22$ (D đang ∞ nên cập nhật $d(D)=22$, $prev=B$). Tới C thử $7 + 10 = 17$, nhưng $d(C)$ hiện là 9 nhỏ hơn nên **giữ nguyên**, không đổi.

Bước 2 — Chốt A. Chọn B ($d=7$), nối lỏng cạnh từ B



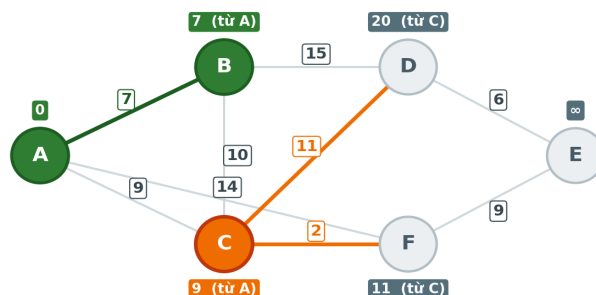
$d(D)=7+15=22$ (mới), $d(C): 7+10=17 > 9 \rightarrow$ giữ nguyên 9

Đỉnh	A	B	C	D	E	F
$d(v)$	0	7	9	22	∞	14
Đỉnh trước	-	A	A	B	-	A
Trạng thái	đã chốt ✓	đã chốt ✓	chờ	chờ	chờ	chờ

Bước 3 — Chốt C

Nhỏ nhất trong {C, D, E, F} là **C** ($d = 9$) → chốt C. Đây là bước thú vị vì xảy ra **hai lần cập nhật lại**: tới D qua C là $9 + 11 = 20$, nhỏ hơn 22 cũ nên $d(D)$ đổi thành 20 ($prev$ đổi từ B sang C); tới F qua C là $9 + 2 = 11$, nhỏ hơn 14 cũ nên $d(F)$ đổi thành 11 ($prev$ đổi từ A sang C).

Bước 3 — Chốt B. Chọn C ($d=9$), nối lỏng cạnh từ C



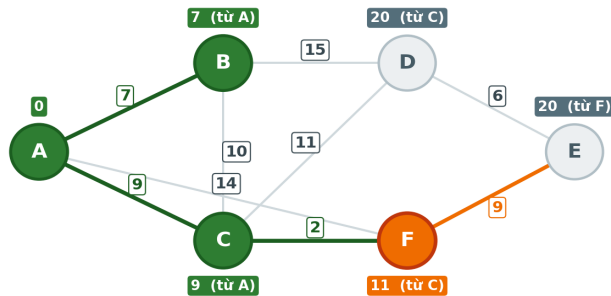
Cập nhật lại! $d(D): 9+11=20 < 22 \checkmark$ $d(F): 9+2=11 < 14 \checkmark$

Đỉnh	A	B	C	D	E	F
$d(v)$	0	7	9	20	∞	11
Đỉnh trước	-	A	A	C	-	C
Trạng thái	đã chốt ✓	đã chốt ✓	đã chốt ✓	chờ	chờ	chờ

Bước 4 — Chốt F

Trong $\{D, E, F\}$, nhỏ nhất là **F** ($d = 11$) $\rightarrow$ chốt F. Từ F nối lỏng sang E: $11 + 9 = 20$. E đang ∞ nên cập nhật $d(E) = 20$, $\text{prev}(E) = F$.

Bước 4 — Chốt C. Chọn F ($d=11$), nối lỏng cạnh từ F



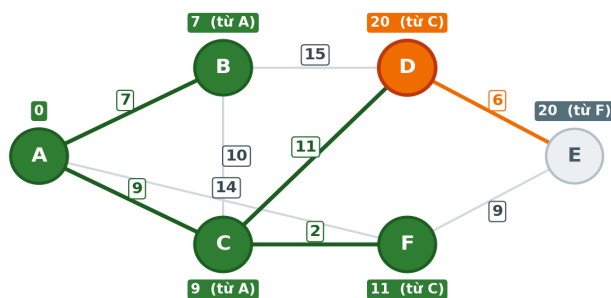
$d(E) = 11 + 9 = 20$ (mới)

Đỉnh	A	B	C	D	E	F
$d(v)$	0	7	9	20	20	11
Đỉnh trước	-	A	A	C	F	C
Trạng thái	đã chốt ✓	đã chốt ✓	đã chốt ✓	chờ	chờ	đã chốt ✓

Bước 5 — Chốt D

Còn lại $\{D, E\}$, cả hai cùng $d = 20$. Khi hòa nhau ta chọn theo thứ tự bảng chữ cái nên chốt **D** trước. Từ D thử sang E: $20 + 6 = 26$, lớn hơn 20 hiện có nên **giữ nguyên**.

Bước 5 — Chốt F. Chọn D ($d=20$), nối lỏng cạnh từ D



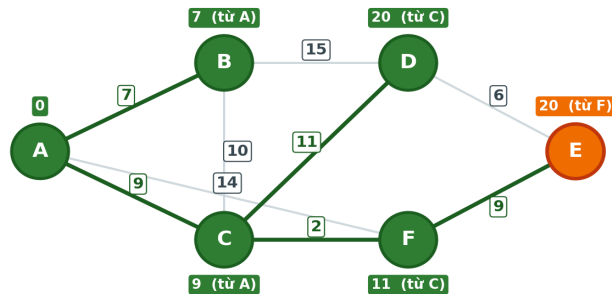
$d(E): 20 + 6 = 26 > 20 \rightarrow$ giữ nguyên 20

Đỉnh	A	B	C	D	E	F
$d(v)$	0	7	9	20	20	11
Đỉnh trước	-	A	A	C	F	C
Trạng thái	đã chốt ✓	đã chốt ✓	đã chốt ✓	đã chốt ✓	chờ	đã chốt ✓

Bước 6 — Chốt E

Chỉ còn **E** ($d = 20$) → chốt E. Mọi đỉnh kề của E đều đã chốt nên không còn gì để nói lỏng. Tất cả các đỉnh đã được chốt → thuật toán kết thúc.

Bước 6 — Chốt D. Chọn E ($d=20$) — đỉnh cuối cùng



E không còn đỉnh kề chưa thăm → Kết thúc

Đỉnh	A	B	C	D	E	F
$d(v)$	0	7	9	20	20	11
Đỉnh trước	-	A	A	C	F	C
Trạng thái	đã chốt ✓	đã chốt ✓	đã chốt ✓	đã chốt ✓	đã chốt ✓	đã chốt ✓

6. Bảng tổng hợp toàn bộ quá trình

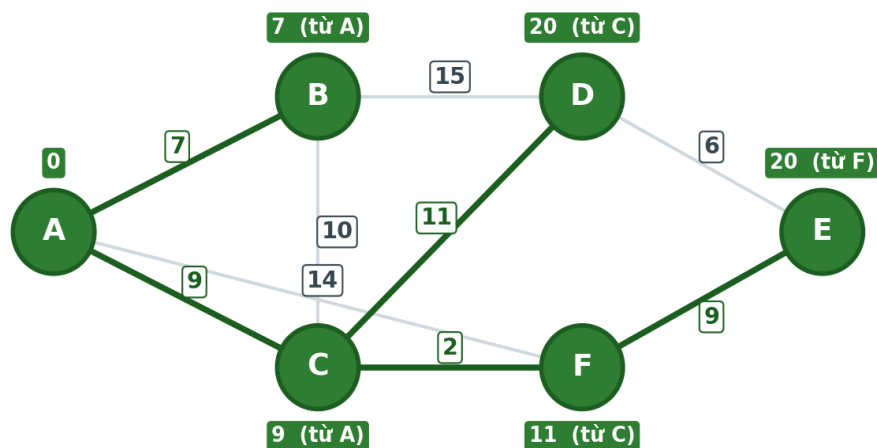
Bảng dưới đây gom tất cả các bước vào một chỗ. Mỗi hàng là một lần chốt; các ô ghi giá trị $d(v)$ sau bước đó. Ô **tô xanh** là giá trị vừa được cập nhật ở bước đó; dấu ✓ đánh dấu đỉnh đã chốt.

Bước	Chốt	A	B	C	D	E	F
Khởi tạo	-	0	∞	∞	∞	∞	∞
Bước 1	A	0✓	7	9	∞	∞	14
Bước 2	B	0✓	7✓	9	22	∞	14
Bước 3	C	0✓	7✓	9✓	20	∞	11
Bước 4	F	0✓	7✓	9✓	20	20	11✓
Bước 5	D	0✓	7✓	9✓	20✓	20	11✓
Bước 6	E	0✓	7✓	9✓	20✓	20✓	11✓

7. Kết quả cuối cùng

Truy ngược theo “đỉnh trước” từ mỗi đỉnh về A, ta dựng lại được đường đi ngắn nhất. Các cạnh tô đậm màu xanh trong hình tạo thành **cây đường đi ngắn nhất**.

Kết quả — Cây đường đi ngắn nhất từ A



Hình 2. Cây đường đi ngắn nhất xuất phát từ A.

Đến đỉnh	Khoảng cách ngắn nhất	Đường đi
B	7	A → B
C	9	A → C
F	11	A → C → F
D	20	A → C → D
E	20	A → C → F → E

8. Ghi nhớ & lỗi thường gặp

✓ Bốn điều phải nhớ

- **Luôn chọn đỉnh có d nhỏ nhất** trong số các đỉnh chưa chốt — đây là “trái tim” của Dijkstra, đừng chọn nhầm đỉnh đã chốt.
- **Đã chốt là không xét lại.** Khi một đỉnh được chốt, $d(v)$ của nó là cuối cùng.
- **Nói lỏng chỉ cập nhật khi ngắn hơn.** Nếu đường mới dài hơn hoặc bằng thì giữ nguyên.
- **Đừng quên ghi “đỉnh trước”,** nếu không sẽ không truy được đường đi, chỉ biết độ dài.

Vì sao Dijkstra cần trọng số không âm? Khi chốt một đỉnh, ta tin rằng không có đường nào ngắn hơn nữa. Điều này chỉ đúng nếu đi thêm cạnh luôn làm quãng đường *tăng lên* (hoặc giữ nguyên). Nếu tồn tại cạnh âm, một đường vòng có thể bất ngờ rẻ hơn, khiến giả định này sai và kết quả không còn chính xác.

Độ phức tạp thời gian: nếu mỗi lần chọn đỉnh nhỏ nhất bằng cách duyệt mảng thì tốn $O(V^2)$; nếu dùng hàng đợi ưu tiên (heap nhị phân) thì giảm xuống $O((V + E) \cdot \log V)$, với V là số đỉnh và E là số cạnh.